TICKET MANAGEMENT SYSTEM (TMS)

MOBILE APPLICATION — COMPREHENSIVE VIVA VOCE & TECHNICAL DEFENSE GUIDE

Project Domain: Enterprise Facility & IT Ticket Management	**Target Platform:** Android & iOS Mobile Application
Frontend Framework: React Native 0.88 + Expo SDK 58	**Backend Architecture:** Node.js + Express.js REST API
Database: MongoDB Atlas Cloud NoSQL (Mongoose ODM)	**Security & Session:** JWT (JSON Web Tokens) + AsyncStorage

1. Executive Summary & Problem Statement

What is TMS Mobile? TMS Mobile is an enterprise-grade cross-platform mobile application designed to digitize, automate, and streamline maintenance tickets, hardware failures, networking disruptions, and civil/electrical complaints across institutional campuses and corporate offices.

Problem Solved: Conventional manual or paper-based complaint registers lead to lost complaints, lack of accountability, delayed technician assignments, no photo verification of issues, and zero analytical visibility for management. TMS Mobile provides an instantaneous 4-tier reporting workflow (Department → Programme → Block → Room) with image capture proofs, instant status tracking, role-based ticket isolation, and real-time administrative intelligence.

2. Languages & Technical Stack Used

The project is engineered strictly following modern mobile development paradigms. Below is the complete technology breakdown:

Category	Technology / Language	Technical Justification / Viva Point
Programming Languages	JavaScript (ES6+) & JSX	Core programming language for mobile logic, async operations (Promises/async-await), and JSX for declarative native UI.
Mobile App Framework	React Native (0.88.0)	Compiles to true native iOS/Android UI components (not a webview). Ensures 60fps high performance and native device API access.
Toolchain & SDK	Expo (SDK 58.0.0)	Streamlines compilation, native asset linking, bundling, and over-the-air testing via Expo Go client.
Navigation Engine	React Navigation v7 Native Stack	Uses native platform navigation primitives for smooth screen transitions, gestures, and zero-flicker native headers.
Client Storage	@react-native-async-storage	Encrypted, asynchronous key-value persistence for offline JWT authentication tokens and persistent user profile caching.
Camera & Media	expo-image-picker	Integrates with native device camera and gallery with base64/URI conversion and MIME validation for ticket photo evidence.

Vector Visualization	react-native-svg	Custom programmatic native SVG rendering for dynamic interactive Donut Charts and category distribution diagrams.
Networking / HTTP	Axios + Interceptors	Handles REST communication, request timeout thresholds, and automatic Bearer JWT injection on every outbound request.
Backend API Runtime	Node.js & Express.js	Non-blocking, event-driven RESTful micro-endpoints handling multipart uploads, role checks, and database aggregation.
Database	MongoDB Atlas (Cloud NoSQL)	JSON-like flexible BSON document model with Mongoose schemas, ObjectId relationship population, and indexed query speeds.

3. Mobile Application Modules (8 Core Modules)

The mobile application is modularized into 8 distinct functional units, each dedicated to a critical operational requirement:

Module Name	Component File	Core Responsibilities & Technical Highlights
1. Splash & Auto-Session Recovery	SplashScreen.js AuthContext.js	Executes on app cold-start. Reads AsyncStorage for existing JWT tokens, fetches user profile from `/api/auth/profile`, and animates pulse logo with native driver while session resolves.
2. Secure Authentication	LoginScreen.js	Email and password input validation with eye-toggle password security. Submits credentials to `/api/auth/login`, saves token locally, and triggers conditional navigation to Dashboard.
3. Role-Aware Dashboard	DashboardScreen.js	Central operational hub. Shows dynamic summary stats (Pending, In Progress, Resolved). For regular users, displays 'My Raised Tickets'; for SuperAdmin, displays 'Total System Tickets'. Includes quick action cards and Profile Modal.
4. 4-Tier Ticket Creation	ComplaintFormScreen.js	Cascading dependent selectors: Department → Programme → Block → Room. Issue type selection chips, description input, and camera/gallery image picker for attachment proof.
5. Role-Isolated Ticket Center	ComplaintsListScreen.js	Multi-tab status filter (All, Pending, Assigned, In-Progress, Closed). Enforces strict Role-Based Access: SuperAdmin views all tickets, Regular Users only see tickets they created, Staff only view assigned tasks.
6. Technician Staff Allocation	ComplaintsListScreen.js (Modal Sub-flow)	SuperAdmin privilege. Opens modal to allocate pending complaints to specialized personnel (Plumber, Electrician, Network Staff, PC Hardware, Software Dev). Updates status to 'Assigned'.
7. Administrative Master Hub	AdminMasterScreen.js	Enterprise administration center with animated navigation drawer to manage institutional entities: Departments, Programmes, Blocks, Rooms, System Roles, and Registered Users.
8. Native Analytics & Reports	ReportsScreen.js	SuperAdmin intelligence dashboard with custom native SVG interactive Donut Charts, resolution rate percentage metrics, category distribution graphs, and volume trends.

4. Complete NPM Packages Matrix (mobile/package.json)

Here is the verified list of production packages running inside the mobile app:

Package Name	Installed Version	Exact Purpose in Mobile App
expo	^58.0.0	Core Expo client runtime and build tools for cross-platform execution.
react-native	0.88.0	Mobile engine providing native components (View, Text, FlatList, Animated).
react	19.3.0	React library powering JSX, Hooks, and Component lifecycle.
@react-navigation/native	^7.0.0 (7.5.1)	Navigation container and route state manager.
@react-navigation/native-stack	^7.0.0 (7.21.0)	Native iOS/Android stack navigator using native screen primitives.
@react-native-async-storage/async-storage	2.2.0	Local unencrypted persistent key-value store for user tokens and cache.
axios	^1.6.8	HTTP client for API REST calls with request/response interceptors.
expo-image-picker	~58.0.10	Native camera and photo library access for defect photographic proof.
expo-status-bar	~58.0.4	Controls device top status bar styling (light/dark content).
react-native-safe-area-context	~5.10.1	Handles screen cutouts, notch areas, and navigation bars on modern devices.
react-native-screens	~4.30.0	Optimizes memory by exposing native UIViewController / Fragment screens.
react-native-svg	15.16.0	Enables SVG rendering for interactive donut charts and vector graphics.
react-native-web	^0.21.4	Allows React Native components to render in web browsers if needed.

5. System Architecture & Ticket Lifecycle

Ticket State Machine: Complaints undergo four distinct lifecycle stages governed by strict role permissions:

Status Stage	Trigger / Action	Authorized Actor	Technical Next Step
1. PENDING	Ticket submitted via mobile form with location and optional photo attachment.	Regular User	Stored in MongoDB with `status: 'Pending'`. Awaits SuperAdmin review.
2. ASSIGNED	SuperAdmin selects appropriate technician from staff directory.	SuperAdmin	`PATCH /api/complaints/:id/assign`. Updates `assignedTo: staffId` and `status: 'Assigned'`.
3. IN-PROGRESS	Assigned technician starts repair work and updates ticket status.	Assigned Staff / Admin	`PATCH /api/complaints/:id/status`. Marks issue under active rectification.
4. CLOSED / RESOLVED	Physical issue solved and verified by technician, user, or SuperAdmin.	Creator / Staff / Admin	Marks `status: 'Closed'`. Included in completed analytics reports.

6. Top 20 Viva Voce Questions & Technical Answers

Study these 20 high-yield questions carefully. Examiners frequently focus on architectural decisions, security, and state management:

Q1: Why did you choose React Native instead of a hybrid Web/WebView approach (like Ionic or Cordova)?

Answer: React Native renders real, native Android and iOS UI widgets (via Native Modules and Fabric renderer) rather than running inside an embedded webview. This provides true 60fps scrolling, fluid animations using the Native Driver, direct access to camera/hardware APIs without lag, and a seamless native user experience.

Q2: What is Expo and why is it used in this project?

Answer: Expo is an open-source framework and platform for React Native. It abstracts native Android/iOS build toolchains, simplifies dependency management, provides battle-tested native APIs (like expo-image-picker and expo-status-bar), and enables instant testing on physical devices via Expo Go.

Q3: Why did you upgrade the project to Expo SDK 58 and React Navigation v7?

Answer: Expo SDK 58 adopted React 19 and React Native 0.88. React 19 introduced architectural breaking changes (such as stricter prop typing, removal of legacy context APIs, and refactoring of ref forwarding). Older React Navigation v6 packages crashed on load under React 19. Upgrading to React Navigation v7 resolved all compatibility bottlenecks and ensured zero runtime crashes.

Q4: How does user authentication work in TMS Mobile?

Answer: We implement stateless JWT (JSON Web Token) authentication. When the user enters their email and password on LoginScreen, the credentials are sent via POST to `/api/auth/login`. The server verifies the password against a bcrypt salt-hash in MongoDB and returns a signed JWT containing the user's ID and role. The mobile app stores this token in `@react-native-async-storage/async-storage`.

Q5: How does the mobile app attach the JWT token to subsequent API requests?

Answer: Through an Axios Request Interceptor defined in `mobile/src/config/api.js`. Before every outgoing HTTP request, the interceptor asynchronously retrieves the token from AsyncStorage and injects it into the HTTP headers as `Authorization: Bearer `. The backend middleware verifies this token before allowing route execution.

Q6: How is Role-Based Access Control (RBAC) enforced between SuperAdmin and Regular Users?

Answer: RBAC is enforced at two distinct levels: 1) Backend Level: In `complaintController.js`, `listComplaints` checks `req.user.role`. If SuperAdmin, it returns all records. If a regular user, the MongoDB query is strictly filtered to `{ createdBy: requester.id }`. 2) Mobile UI Level: In `ComplaintsListScreen.js`, client-side filters further verify user ownership, and action buttons like 'Assign Staff' are conditionally rendered only for SuperAdmin.

Q7: Explain the Cascading Dropdown mechanism in ComplaintFormScreen.

Answer: The screen uses a 4-level dependent selection hierarchy: Department → Programme → Block → Room. When a user selects a Department, an effect hook fetches programmes linked to that `departmentId`. Selecting a programme loads corresponding blocks, and selecting a block dynamically filters available rooms. This eliminates human error and guarantees location accuracy.

Q8: How are image attachments uploaded from the mobile device to the server?

Answer: When the user selects or snaps a photo using `expo-image-picker`, the app receives a local file URI. The app constructs a standard `FormData` object containing the image URI, MIME type (`image/jpeg`), and generated filename along with ticket metadata. It is sent via POST to `/api/complaints` with `Content-Type: multipart/form-data`, where Multer middleware saves it to the backend `uploads/` directory.

Q9: What is React Context API and why was it used instead of Redux?

Answer: React Context API (`AuthContext.js`) provides centralized global state without prop-drilling. For this application's state requirements (user profile, authentication status, login/logout actions), Context API is lightweight, built directly into React, requires zero external boilerplate, and does not incur the heavy memory footprint of Redux.

Q10: What is AsyncStorage and what are its limitations?

Answer: AsyncStorage is an asynchronous, unencrypted, persistent key-value storage system for React Native. We use it to store the JWT string and basic session flags. Its limitation is that it is not intended for large datasets or highly sensitive unencrypted secrets, which is why we only store the temporary auth token, not passwords.

Q11: How does the mobile app draw graphs in ReportsScreen without external charting libraries?

Answer: We built a custom SVG Donut/Pie chart component using `react-native-svg`. It calculates angles using trigonometric formulas (`Math.cos` and `Math.sin`), generating SVG arc paths (`d='M... A...'`) for each complaint category slice. This guarantees zero bloat, native vector rendering, and responsive scaling across any device resolution.

Q12: How are passwords secured in the database?

Answer: Passwords are never stored in plaintext. In `authController.js`, we use `bcryptjs` with 10 salt rounds (`bcrypt.hash(password, 10)`). During login, `bcrypt.compare()` compares the plain credential against the one-way hash, preventing credential theft even in the event of database inspection.

Q13: How does the mobile app communicate with the local development backend?

Answer: In `mobile/src/config/api.js`, the base URL is configured with the host computer's Local Area Network IP address (e.g., `http://10.249.68.142:5000/api`). This allows both the physical mobile device and the computer to talk over the same Wi-Fi subnet without relying on internet tunneling.

Q14: What is the purpose of `useNativeDriver: true` in your animations?

Answer: In `SplashScreen.js` and `LoginScreen.js`, `useNativeDriver: true` offloads animation calculation (such as opacity and scale) from the single-threaded JavaScript thread to the native UI thread (RenderThread). This ensures that even if the JS thread is busy loading data, the logo pulse animation remains perfectly smooth.

Q15: What Mongoose models exist in the backend supporting the app?

Answer: There are 7 Mongoose schemas: `User` (credentials, role, department), `Complaint` (location, type, remarks, photo, status, createdBy, assignedTo), `Department`, `Programme`, `Block`, `Room`, and `Role`.

Q16: How does the application handle error handling and network failures?

Answer: Axios calls are wrapped in `try/catch` blocks. In `api.js`, timeout is set to 10000ms. If a network failure occurs, the UI displays informative `Alert.alert` dialogs, catch fallbacks prevent UI crashes, and pull-to-refresh (`RefreshControl`) allows immediate retries.

Q17: What is the difference between `useEffect`, `useState`, and `useMemo` in your screens?

Answer: `useState` manages local reactive state (e.g., entered text, ticket list). `useEffect` handles side-effects (e.g., fetching complaints on screen load or filter change). `useMemo` memoizes expensive calculations (e.g., summing ticket totals for SVG chart slices so it doesn't recalculate on every render).

Q18: What is `react-native-safe-area-context` and why is it essential?

Answer: Modern smartphones feature screen notches, dynamic islands, rounded display corners, and bottom gesture bars. `react-native-safe-area-context` programmatically detects these hardware insets and provides padding so UI elements (buttons, inputs) are never clipped or obscured.

Q19: Can a SuperAdmin raise a complaint in your system?

Answer: By design, SuperAdmins are restricted from raising complaints; their duty is supervisory, administrative, and allocation-focused. If a SuperAdmin attempts to raise a complaint, the backend controller rejects the request with HTTP 403 ('Super Admin cannot raise complaints'). Regular users raise tickets.

Q20: What are 3 prominent future enhancements you would add to TMS Mobile?

Answer: 1) Push Notifications via Expo Notifications / Firebase Cloud Messaging (FCM) to alert users when a ticket status changes. 2) Real-time technician live chat via WebSockets (Socket.io). 3) In-app Barcode / QR Code scanning on classroom doors to instantly autofill block and room numbers.

7. Quick Viva Morning 'Cheat Sheet' (Key Buzzwords)

Topic	Key Technical Phrases to Speak to the Examiner
Frontend Architecture	• Component-driven React Native architecture • Declarative JSX, Context API state management • Zero-webview, compiled directly to native Android/iOS primitives
Security & Auth	• Stateless JSON Web Token (JWT) architecture • Password hashing with BCrypt (10 rounds) • Axios request interceptors inject Bearer token automatically • Persistent session management via AsyncStorage
Data Isolation & RBAC	• Dual-layer Role Based Access Control (Server query filter + Client UI filter) • SuperAdmin = Global visibility & Staff assignment • Regular User = Isolated view of own submitted tickets (`createdBy == req.user.id`) • Staff = Restricted to assigned maintenance issues
Engineering Challenge Solved	• Upgraded to Expo SDK 58 & React Navigation v7 to resolve React 19 breaking changes. • Optimized Metro bundler compilation for 994 native modules with zero compilation errors.